Creating FIWARE-compliant diagnostic brokers for digital twins in water research projects

G. Lewis[1*], L. Vamvakeridou-Lyroudia[2], A. S. Chen[3], S. Djordjević[4], D. A. Savić[5]

[1,2,3,4,5] Centre for Water Systems, University of Exeter. Harrison Building, North Park Road, Exeter, EX4 4QF, UK.

[2,5] KWR Water Research Institute. Groningenhaven 7, P.O. Box 1072, 3430 BB Nieuwegein, the Netherlands.

***Corresponding author: E-mail: g.lewis2@exeter.ac.uk**

ABSTRACT

FIWARE is a great piece of technology and forms the data hub of recent digital twin-based research projects that University of Exeter has been involved with (aqua3S, Fiware4Water, LOTUS and Waterverse). FIWARE's http-based interface gives users a small vocabulary of functions to work data through the data CRUD lifecycle to create, retrieve, update, and delete data in comparison to SQL or no-SQL approaches.

However, FIWARE does have some weaknesses that tend to get exposed during development activities rather than operations, which can create a lot of issues for developers. FIWARE's http-based approach can make it difficult to onboard large amounts of historic data into a broker in a timely manner, something which can be essential when looking to test algorithms that require large amounts of historic broker data.

The purpose of this paper is to document an approach taken to implementing FIWARE ngsi-ld/v1 functionality to make it more suitable for the development and testing of digital twins, rather than their operation.

Keywords: FIWARE, digital twins, Python, SQL

1 Purpose

FIWARE [1] is a great piece of technology and forms the data hub of recent digital twin-based research projects that University of Exeter has been involved with (aqua3S [2], Fiware4Water [3], LOTUS [4] and Waterverse [5]). FIWARE's http-based interface gives users a small vocabulary of functions to work data through the data CRUD lifecycle to create, retrieve, update, and delete data in comparison to SQL or no-SQL approaches.
However, FIWARE does have some weaknesses that tend to get exposed during development activities rather than operations, which can create a lot of issues for developers.

The aqua3S project used Orion-LD/Cygnus [6] brokers which proved challenging to integrate reliably into the project, causing issues for developers relying on broker-based data. Whilst Stellio [7] was used for some early development, its lack of support for multiple tenants and large memory footprint made it difficult to work with across multiple projects and environments.
Once Orion-LD was working reliably, it proved difficult to load the broker up with historic data for testing, with each update requiring a HTTP patch request. For a pilot of 20 sensors collecting data every 15 minutes, a year's backlog would require about 700.000 requests, something which is well-beyond the HTTP standard to deliver in a reasonable timeframe.

Likewise, whilst the Stellio broker used in Fiware4Water worked flawlessly during the project, its large runtime memory requirements made it impossible to run on lightweight test servers and, like the Orion-LD HTTP patch issues, it was difficult to patch or get many requests to and from the broker.

Finally, the LOTUS project required a broker for the Windows platform and, even working through Docker and WSL2 [8], existing brokers were not recommended.

The purpose of this paper is to document an approach taken to implementing FIWARE ngsi-ld/v1 functionality [9] to make it more suitable for the development and testing of digital twins, rather than their operation.

2 Materials and Methods

2.1 FIWARE broker

FIWARE brokers are implementations of CMI-based standards [9] for ngsi-ld/v1 or ngsi-ld/v2, describing a CRUD-based data lifecycle of create, retrieve, update, and delete for smart models, realised as JSON-LD key-value pairs. FIWARE maintains a repository of smart models [10], that developers can use, extend or re-implement for their projects, The broker interface is realised through HTTP requests using post-based requests for smart model creation, get for retrieval, patch to update, and delete for deletion.

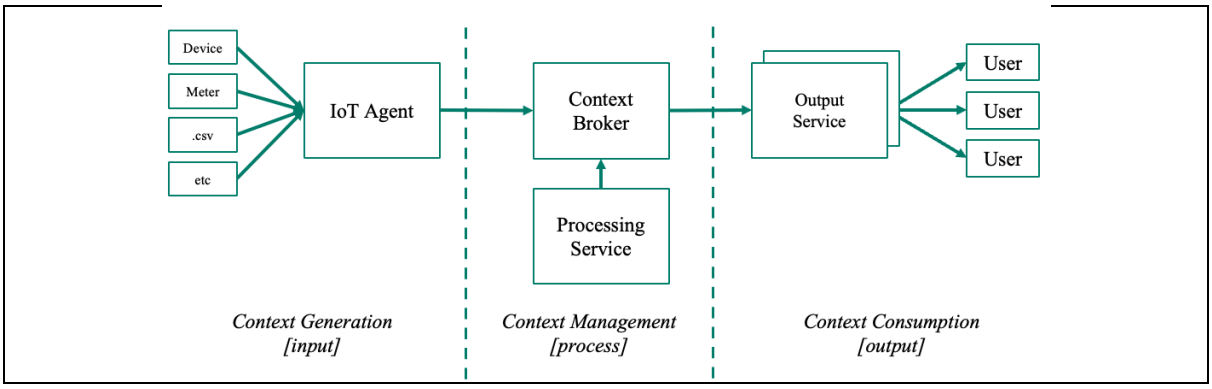

Figure 1 Prototypical FIWARE architecture

Figure 1 outlines a prototypical FIWARE implementation with three core functions: context generation, context management and context consumption. For digital twins, real-world data is collected as inputs to the IoT Agent in the context generation phase. The goal of this phase

is to collect data and create / generate smart model instance (contexts) representations of the data to be stored in the broker.

The context management function of the broker describes functionality where contexts can be processed to create new data and/or contexts. Typically, this describes activities where 'dumb' data is collected from the IoT Agent and then processed, potentially with other data, to create insights, e.g., sensor data may be collected by the IoT Agent and is processed later to determine is the values received are in or out of range.

Finally, the context consumption function describes activities where broker data is processed as output. This is likely to be situation specific and may include activities like producing daily reports, providing data for webapps and so on.

2.2 Development

The existing broker interface was captured by sending calls to a Stellio broker determine correct responses and returned payloads and examining the ngsi-ld/v1 specification. This interface was built out using Python/Flask to mock functionality and SQLite was used to build the underlying broker data model, resulting in the logical architecture, Figure 2.

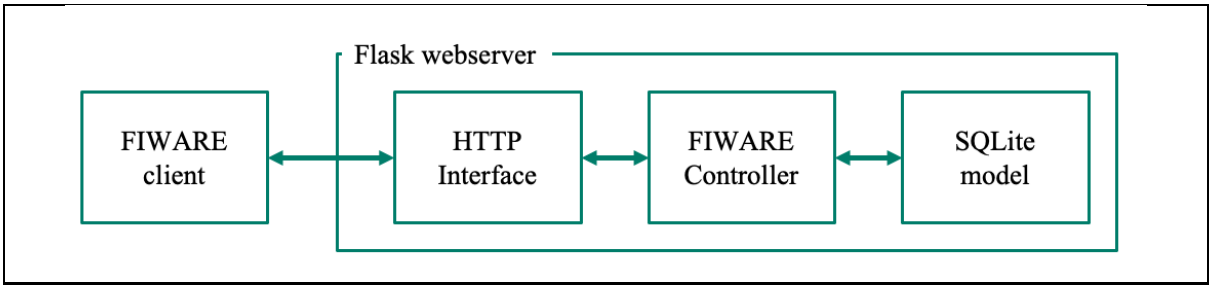

Figure 2 Development broker, logical architecture

2.3 Smart model management

Smart models were stored as SQL tables, initially with each FIWARE model entity stored as separate table using the entity's unique ID and tenant to form a unique table name. The entity table contained an SQL ID as a unique reference, timestamp, and smart model as text. For working with temporal data, the 'obersevedAt' element, models were parsed on table creation with temporal components stored as separate tables to facilitate historic data management.

Figure 3 details the process, with a smart model instance on the left. Upon adding to the context broker through a PUT request, the smart model is entered into the *stream_table*, storing the tenant, model type and model id, which can be used to create the table label root of <tenant>_<type>_<id> as SQL sanitised data. The model instance content is stored in a unique <tenant>_<id> table, as *aaa_urn_ngsi_ld_device_mydevice*, with the data field storing the model instance as dumb text and no context validation is performed.
The temporal 'pressure' attribute is stored in a separate unique table of <tenant>_<type>_<id>_<attribute>, *aaa_urn_ngsi_ld_device_mydevice_pressure*.

Smart model instance

```
{
    "@context": "https://schema.lab.fiware.org/ld/context",
    "id": "urn:ngsi-ld:Device:MyDevice",
    "type": "Device"

    "name": {
    "type": "Property",
    "value": "P12"
    },

    "pressure": {
    "observedAt": "2022-05-12T08:26:17Z",
    "type": "Property",
    "unitCode": "N23",
    "value": "52.534"
    }
}
```

Tenant	Type	id
AAA	Device	urn:ngsi-ld:Device:MyDevice

stream_table

ID	Timestamp	data
1	12345678	{"@context":"https://schema.lab.fiwar e...

aaa_urn_ngsi-ld_device_mydevice

ID	Timestamp	data
1	12345678	{"pressure", 'observedAt":'2022-05-12T08..
2	12345678	{"pressure", 'observedAt":'2022-05-12T08..

aaa_urn_ngsi-ld_device_mydevice_pressure

Figure 3 smart model to table conversions

For each patch request, the relevant smart model table was retrieved and updated to reflect the changes request in the call. For non-temporal data, e.g., patching the *name* attribute will result in the *aaa_urn_ngsi_ld_device_mydevice* data field being updated to reflect the value of the name attribute. This is achieved through an SQL update call, i.e., no new records are added to the table.

In the case of temporal data, i.e., the *pressure* attribute with an *obersevedAt* element, the *aaa_urn_ngsi_ld_device_mydevice* table is updated in the same way as the non-temporal case above, but in addition, the *aaa_urn_ngsi_ld_device_mydevice_pressure* table is updated by performing an SQL insert to add a new record to the existing record set, thus creating the temporal collection of data.

2.4 Patch array

To address the performance issues of multiple patch requests, a new HTTP request was developed that took an array of patch entries as its payload. That way, a single HTTP request could trigger thousands of patches on the server-side.
This was realised by adding a new end point to the HTTP interface and re-using existing functionality in the FIWARE controller, Figure 2, to process each element in the array passed into the end point. The success of this approach (with minimal refactoring to the existing code base) demonstrates that the performance penalty of multiple HTTP patch requests was due to the HTTP overhead rather than processing patch data in the broker.

2.5 Broker archiving

Archiver functionality was developed to dump broker contents and re-build a broker from dumped data. Whilst broker content could be easily extracted from a *normal* broker, by retrieving all the instances of a given smart model type and then retrieving associated temporal data, having the patch array functionality made it possible to rebuild a broker from large scale temporal data. In addition, a request was developed that would return all smart model IDs, by tenant and type. This was a huge benefit as it made it straightforward to view the current state of the broker.

2.6 Broker operations environments

The resulting broker was used as a Python application and as a Docker image, with much of the aqua3S visualisation development undertaken with the broker application and tested with the Docker image.

3 Results and Discussion

The development broker has been used successfully on several projects and across all three DevOps environments: bench development, remote testing, and operations, Figure 4.

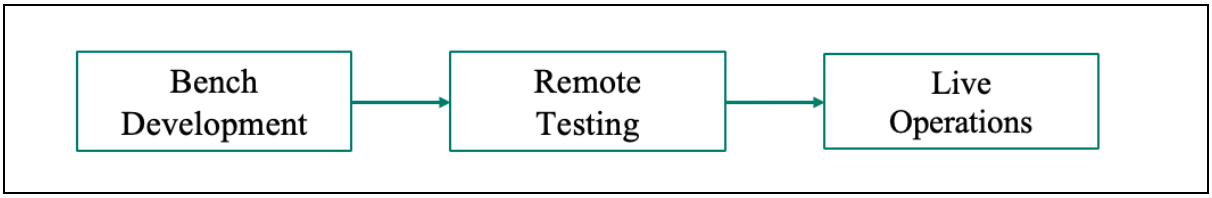

Figure 4 Typical webapp DevOps lifecycle

3.1 Development-side activities

For initial bench development, the broker made it possible to load large amounts historic data in a timely manner, which was essential for work with large-scale data development and testing on each project. Running the broker as an application made it possible to step through both broker and client application functionality to trace error conditions and add appropriate diagnostics. It was also trivial to run multiple brokers across multiple projects concurrently.

For the remote testing environment, the development broker was packaged as a Docker image and ran as part of a test applications for months on a lightweight remote testbed (4Gb RAM). Typically, the test server ran multiple Docker containers for the aqua3S, Fiware4Water and LOTUS projects.

3.2 Operations-side activities

Ironically, the development broker was used in the operations environment of aqua3S although it was not intended to. Due to issues with adding temporal data to Cygnus, much of the aqua3S early warning and visualisation smart models were left on the development broker until the final prototype of the project, that included a substantial refactor of smart models. The development broker operated as expected and did not cause any noticeable issues.

3.3 Patch limitations

Whilst the approach of using the 'patch array' is highly attractive for uploading large amount of historic data, there are potential issues, particularly where data is to be processed on reception, either through FIWARE's subscription model or HTTP delegates, i.e., making an HTTP request on receiving data. This effectively pushes the HTTP bubble downstream, though it is possible to disable delegate calls for 'dumb' data uploading.

There is a tendency to look to extend the functionality of the 'patch array' to a dictionary or even writing directly to the database, though it is hard to recommend these approaches.

4 Conclusions

On the surface 'reinventing the wheel' is a terrible idea. However, it's been an incredibly useful tool for the niche issues that were encountered, given me a much better / deeper understanding of brokers and how to make them work on projects and opens the door for interesting extensions, e.g., sockets vs, HTTP. However, Stellio and Orion-LD are good products and should be used where possible.

4.1 Archiving / De-archiving brokers

One of the big challenges with working with brokers has been in archiving and de-archiving brokers. Whilst archiving is relatively straightforward, in that an algorithm that collects all instances of a given model type and collects temporal data will effectively create an archive, the limitations of HTTP patch requests can leave users with no way of de-archiving content.
During the development of aqua3S, it was very useful to archive the production broker and re-create it in development environments and this was relatively trivial with the diagnostic broker.

Likewise, the LOTUS project has proposed functionality for selective temporal archiving on brokers, i.e., to remove data between certain dates. Currently, there would appear to be no functionality in the ngsi-ld/v1 (or ngsi-ld/v2), but this is something that could be developed.

4.2 Limitations and opportunities

The diagnostic broker has clear limitations, it has not been designed as an operational broker and shouldn't be used as such, it's not FIWARE compliant and it ignores the @context flag.

However, this approach to prototype broker development does offer opportunities to develop functionality that is not currently supported by FIWARE, or part of current FIWARE roadmaps. Selective archiving and TCP/IP-based communications are interesting areas to consider.

5 Acknowledgements

The work presented in this paper was funded by the ongoing EC H2020 LOTUS (GA 820881) and Waterverse (GA 101070262) projects and completed aqua3S (GA 832876) and Fiware4Water (GA 821036) projects.

6 References

[1] "FIWARE Homepage," [Online]. Available: https://www.fiware.org/. [Accessed 9 March 2023].

[2] "aqua3S Project Homepage," [Online]. Available: https://aqua3s.eu/. [Accessed 9 March 2023].

[3] "Fiware4Water Project Homepage," [Online]. Available: https://www.fiware4water.eu/. [Accessed 9 March 2023].

[4] "LOTUS Project Homepage," [Online]. Available: https://www.lotus-india.eu. [Accessed 9 March 2023].

[5] "Waterverse Project Homepage," 9 March 2023. [Online]. Available: https://waterverse.eu/.

[6] "FIWARE Orion-LD Docker Image," [Online]. Available: https://github.com/FIWARE/context.Orion-LD. [Accessed 9 March 2023].

[7] "Stellio Docker Image," [Online]. Available: https://github.com/stellio-hub/stellio-context-broker. [Accessed 9 March 2023].

[8] Microsoft, "Windows Subsystem for Linux Documentation," [Online]. Available: https://learn.microsoft.com/en-us/windows/wsl/. [Accessed 9 March 2023].

[9] "ETSI Context Information Management (CIM); NGSI-LD API," [Online]. Available: https://www.etsi.org/deliver/etsi_gs/CIM/001_099/013/01.01.01_60/gs_CIM013v010101p.pdf. [Accessed 9 March 2023].

[10] "FIWARE Smart Data Models," [Online]. Available: https://smart-data-models.github.io/data-models/. [Accessed 15 March 2023].